

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-
a075115e9de0a29eb.jsonl`
- SHA-256: `a681dd7073bd010dc2c2c0b71272321c14b20cf0dec9f8e4116b118798139d22`
- Source modified: `2026-06-12T10:46:26+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:43:40.277Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refuted if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [medium] src==out same-file guard is a case-sensitive string compare ? on Windows, --force + a case-variant path makes ffmpeg -y truncate the SOURCE video (C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:670-675)

`os.path.abspath(out) == os.path.abspath(src)` does not normalize case (or resolve junctions/subst drives/8.3 names). On the project's actual platform (win32, NTFS case-insensitive), `--out data\videos\MATCH.MP4` vs `--video-path data\videos\match.mp4` passes the guard. The next check (`os.path.exists(out)`) IS case-insensitive, so it fires and tells the user to 'pass --force to overwrite' ? actively steering them into the destructive path. With --force, ffmpeg opens the same physical file for read and write (`-y`, no prompt; ffmpeg's own in==out detection is also exact-string), destroying the irreplaceable original recording. ffmpeg only protects against exact-string matches.

Proposed fix: Compare os.path.normcase(os.path.realpath(out)) == os.path.normcase(os.path.realpath(src)), and additionally use os.path.samefile(out, src) when out exists (catches hardlinks/junctions).

2. user (2026-06-12T10:43:47.809Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)

a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)

```
docs/INGESTION_PIPELINE.md | 29 ++++
src/cli/curate.py           | 147 ++++++
```

```
src/core/proxy.py          | 242 ++++++
tests/test_proxy.py        | 309 ++++++
4 files changed, 724 insertions(+), 3 deletions(-)
```

3. user (2026-06-12T10:43:48.324Z)

feat/prep-annotation-proxy

4. user (2026-06-12T10:43:54.361Z)

```
600             writer.writerow(["start", "end"])
601             for start, end in intervals:
602                 writer.writerow([start, end])
603
604             manifest.append({
605                 "video_id": video.video_id,
606                 "sport": sport.value,
607                 "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
608                 "local_path": video.local_path,
609                 "video_url": video.video_url,
610                 "match_name": video.match_name,
611                 "fps": video.fps,
612                 "resolution": video.resolution,
613                 "license_type": video.license_type.value,
614                 "is_commercial": video.is_commercial,
615                 "status": video.status.value,
616                 "num_segments": len(intervals),
617             })
618
619         os.makedirs(out_dir, exist_ok=True)
620         manifest_doc = {
621             "commercial_only": commercial_only,
622             "statuses": [s.value for s in statuses],
623             "total_videos": len(manifest),
624             "total_segments": sum(m["num_segments"] for m in manifest),
625             "eval_sets": manifest,
626         }
627         manifest_path = os.path.join(out_dir, "manifest.json")
628         with open(manifest_path, "w", encoding="utf-8") as f:
629             json.dump(manifest_doc, f, indent=2)
630
631         print(f"\n=== Exported eval/training set to '{out_dir}' ===")
632         if manifest:
633             summary = [[m["sport"], m["video_id"], m["num_segments"],
634                 "YES" if m["is_commercial"] else "NO",
635                 os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
636                 for m in manifest]
637             print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
638             print(f"\nVideos exported : {len(manifest)}")
639             print(f"Total segments : {manifest_doc['total_segments']}")
640             if skipped_noncomm:
641                 print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
642             if skipped_empty:
643                 print(f"Skipped (no annotations) : {skipped_empty}")
644             print(f"Manifest : {manifest_path}\n")
645
646     def prep_annotation_proxy(self, args):
647         """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648
649         The proxy is what the annotation vendor receives: downscaled and
650         scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651         but with the source's EXACT frame count / fps / timeline, so frame N
```

```

652 on the proxy is frame N on the original and returned labels apply to
653 the original unchanged. Parity is verified after the encode; a proxy
654 that fails verification is deleted, never shared. Provenance (source +
655 proxy MD5s) is appended to the proxy manifest ? downstream processing
656 (import-local, export, the indexer) keeps running on the ORIGINAL file.
657 """
658 from src.core import proxy as proxylib
659
660 src = args.video_path
661 if not os.path.exists(src):
662     print(f"Error: video '{src}' not found.")
663     sys.exit(1)
664
665 stem = os.path.basename(src).split('.')[0]
666 # Mirror import-local's video_id derivation so the manifest row and a
667 # later DB registration of the same source agree on the key.
668 video_id = args.video_id or stem.lower().replace(" ", "_")
669 out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670 if os.path.abspath(out) == os.path.abspath(src):
671     print("Error: --out must differ from --video-path.")
672     sys.exit(1)
673 if os.path.exists(out) and not args.force:
674     print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675     sys.exit(1)
676
677 try:
678     src_info = proxylib.probe_video(src)
679 except proxylib.ProxyError as e:
680     print(f"Error: {e}")
681     sys.exit(1)
682 if src_info.avg_fps is None:
683     print(f"Error: could not determine fps of '{src}' ? the frame<->time "
684           f"contract needs it.")
685     sys.exit(1)
686 if proxylib.is_vfr(src_info) and not args.allow_vfr:
687     print(f"Error: '{src}' looks variable-frame-rate "
688           f"r={src_info.r_fps} vs avg={src_info.avg_fps}.")
689     print(" The vendor pipeline recomputes times as frame/fps, which is only "
690           "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691           "proceed anyway (timestamps stay identical to the original, but "
692           "frame/fps seconds are approximate for BOTH files).")
693     sys.exit(1)
694
695 os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
696 cmd = proxylib.build_proxy_command(src, out, height=args.height,
697                                   crf=args.crf, gop=args.gop,
698                                   preset=args.preset)
699 print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
700       f"fps {src_info.avg_fps:g} preserved)...")
701 try:
702     proxylib.run_ffmpeg(cmd)
703     proxy_info = proxylib.probe_video(out)
704 except proxylib.ProxyError as e:
705     print(f"Error: {e}")
706     sys.exit(1)
707
708 errors = proxylib.verify_parity(src_info, proxy_info)
709 if errors:
710     print("Error: proxy FAILED frame<->time parity verification:")
711     for err in errors:
712         print(f" - {err}")
713     try:
714         os.remove(out)
715         print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
716               f"poisons golden labels).")

```

```

717         except OSError:
718             print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
719                   f"share it with the vendor.")
720         sys.exit(1)
721
722     print("Hashing source + proxy (provenance)...")
723     source_md5 = proxylib.md5_file(src)
724     proxy_md5 = proxylib.md5_file(out)
725
726     from datetime import datetime, timezone
727     proxylib.append_manifest_row(args.manifest, {
728         "video_id": video_id,
729         "source_path": os.path.abspath(src),
730         "source_md5": source_md5,
731         "proxy_path": os.path.abspath(out),
732         "proxy_md5": proxy_md5,
733         "fps": f"{src_info.avg_fps:g}",
734         "frame_count": src_info.nb_frames,
735         "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
else "",
736         "source_resolution": src_info.resolution,
737         "proxy_resolution": proxy_info.resolution,
738         "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
739                             f"preset={args.preset} height<={args.height} "
740                             f"fps_mode=passthrough audio=aac160k"),
741         "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
742     })
743
744     src_mb = os.path.getsize(src) / (1024 * 1024)
745     out_mb = os.path.getsize(out) / (1024 * 1024)
746     print("\n=== Annotation proxy ready ===")
747     print(f"  Video ID      : {video_id}")
748     print(f"  Source         : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
{source_md5})")
749     print(f"  Proxy          : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
{proxy_md5})")
750     print(f"  Parity         : VERIFIED ? {src_info.nb_frames} frames @
{src_info.avg_fps:g} fps on both")
751     print(f"  Manifest       : {args.manifest}")
752     print("\nNext:")
753     print("  1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
754         "box of every rally on the exact contact/dead frame).")
755     print("  2. On delivery, convert-cvat may use --video-path with either file "
756         "(identical fps/frame count by construction).")
757     print("  3. import-local / export keep pointing at the ORIGINAL file ? the "
758         "indexer keys videos by its MD5.\n")
759

```

5. assistant (2026-06-12T10:44:18.032Z)

Now let me read the new proxy engine to check the ffmpeg command construction.

6. user (2026-06-12T10:44:18.170Z)

```

1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the ±0.3 s boundary bar.
7      The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8      downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9      so frame N on the proxy IS frame N on the original and returned labels apply to
10     the original unchanged.
11

```

```

12 The ADR-002 proxy contract enforced here:
13 - video timestamps pass through untouched (``-fps_mode passthrough``, never
14 ``-r``): same frame count, same fps, no trim, no leading padding;
15 - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16 failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17 golden labels;
18 - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19 survives the handoff (the indexer keys videos by file MD5; downstream
20 processing must keep running on the ORIGINAL file).
21 """
22 import csv
23 import hashlib
24 import os
25 import shutil
26 import subprocess
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29
30
31 class ProxyError(RuntimeError):
32     """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35 #: Columns of the provenance manifest (one row appended per generated proxy).
36 MANIFEST_FIELDS = [
37     "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38     "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39     "encoder_settings", "created_at",
40 ]
41
42 # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43 # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44 # back-stepping; CRF 19 keeps the ball/shuttle legible.
45 DEFAULT_HEIGHT = 1080
46 DEFAULT_CRF = 19
47 DEFAULT_GOP = 15
48 DEFAULT_PRESET = "medium"
49
50
51 @dataclass
52 class ProbeInfo:
53     """The frame?time identity of one video stream, as ffprobe reports it."""
54     width: Optional[int] = None
55     height: Optional[int] = None
56     avg_fps: Optional[float] = None # avg_frame_rate (frames / duration)
57     r_fps: Optional[float] = None # r_frame_rate (the container's base rate)
58     nb_frames: Optional[int] = None # demuxed packet count == frame count
59     duration: Optional[float] = None # video stream duration, else container
60
61     @property
62     def resolution(self) -> str:
63         if self.width and self.height:
64             return f"{self.width}x{self.height}"
65         return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None
72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:

```

```

77         return None
78     return num_f / den_f
79     return float(value)
80 except ValueError:
81     return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffprobe (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffprobe must stop it, not degrade it.
91     """
92     if not shutil.which("ffprobe"):
93         raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffprobe", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106         ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffprobe failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:
117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)
125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's
129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),
137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )

```

```

142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                        crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                        preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
194     """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []
202
203     if src.nb_frames is None or proxy.nb_frames is None:
204         errors.append("frame count unavailable on source and/or proxy ? cannot prove
205 parity")
206     elif src.nb_frames != proxy.nb_frames:

```

```

206         errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208         if src.avg_fps is None or proxy.avg_fps is None:
209             errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210         elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211             errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213         if src.duration is None or proxy.duration is None:
214             errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215         else:
216             tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217             if abs(proxy.duration - src.duration) > tol:
218                 errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                             f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221         return errors
222
223
224     def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225         """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226         digest = hashlib.md5()
227         with open(path, "rb") as f:
228             for chunk in iter(lambda: f.read(chunk_size), b""):
229                 digest.update(chunk)
230         return digest.hexdigest()
231
232
233     def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234         """Append one provenance row, writing the header only on first creation."""
235         parent = os.path.dirname(os.path.abspath(manifest_path))
236         os.makedirs(parent, exist_ok=True)
237         is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238         with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239             writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240             if is_new:
241                 writer.writeheader()
242             writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

7. user (2026-06-12T10:44:52.558Z)

```

79:         "python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
92:         print(" -> pass --force to overwrite, or use a distinct --video-id to keep
both.")
95:         print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
409:         and resumable; `--force` re-fetches everything.
456:         # Skip if we already have a good-enough local file (unless --force).
646:     def prep_annotation_proxy(self, args):
671:         print("Error: --out must differ from --video-path.")
674:         print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
755:         print(" 2. On delivery, convert-cvat may use --video-path with either file "
811:             f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
816:         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
817:         CSV (--csv). Writes, into --out-dir:
1090:     import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
1097:     import_parser.add_argument("--force", action="store_true",
1111:                             help="Skip videos already at/above this height unless --force
(default: 720)")
1114:     fetch_parser.add_argument("--force", action="store_true",

```



```

1136:         "prep-annotation-proxy",
1140:     prep_parser.add_argument("--video-path", required=True,
1142:     prep_parser.add_argument("--out",
1162:     prep_parser.add_argument("--force", action="store_true",
1171:     cvat_parser.add_argument("--out", required=True,
1173:     cvat_parser.add_argument("--video-path",
1186:     vd_parser.add_argument("--input", help="CVAT XML export (use with --video-path/--fps)")
1188:     vd_parser.add_argument("--video-path",
1192:     vd_parser.add_argument("--out-dir", default="data/validation",
1198:     export_parser.add_argument("--out-dir", default="data/eval_export",
1225:     elif args.command == "prep-annotation-proxy":
1226:         cli.prep_annotation_proxy(args)

```

8. user (2026-06-12T10:44:53.277Z)

```

67:     # Unknown rates are treated as VFR (conservative: force the explicit flag).
197:         allow_vfr=False, force=False,

```

9. user (2026-06-12T10:45:05.289Z)

```

abspath equal: False
exists(case-variant): True
normcase+realpath equal: True

```

10. user (2026-06-12T10:45:07.287Z)

```

Name      Length
----      -
match.mp4 13089

```

11. user (2026-06-12T10:45:20.755Z)

```

[out#0/mp4 @ 0000014d2adce800] Codec AVOption b:a (set bitrate (in bits/s)) has not been used
for any stream. The most likely reason is either wrong type (e.g. a video option with no video
streams) or that it is a private option of some decoder which was not actually used for any
stream.
frame= 60 fps=0.0 q=-1.0 Lsize=          26KiB time=00:00:01.93 bitrate= 109.3kbits/s speed=41.8x
elapsed=0:00:00.04
exit=0 before=13089 after=26408

```

12. user (2026-06-12T10:45:57.469Z)

```

samefile: True
Name      Length
----      -
match.mp4 26408

```

13. assistant (2026-06-12T10:46:10.222Z)

The claim is fully confirmed by live reproduction. Calling StructuredOutput.

14. user (2026-06-12T10:46:22.532Z)

Structured output provided successfully

15. assistant (2026-06-12T10:46:26.209Z)

Verification complete ? the claim is confirmed as real (medium severity), with the destructive path reproduced end-to-end on a scrap file.